

Product Requirements Document

02 — Product Requirements Document (PRD)

Last updated: 2026-06-08

1. Product summary

Productivity Shastra is an internal productivity suite that unifies several standalone tools behind one login and one dashboard ("hub"). Each tool helps a user audit, plan, and improve how they spend their time and run their work.

2. Goals

- Give users **one place** (one login, one dashboard) to access every tool.
- Preserve each tool's existing functionality exactly (no feature regressions).
- Provide a clean, consistent **hub** → **tool** → **back to hub** navigation.
- Be **deploy-ready** as a static frontend now, and **backend-ready** later without re-architecting.

3. Non-goals (for the current phase)

- No real backend / database yet (auth and data are local-only by design).
- No cross-tool data sharing yet (each tool owns its own local data).
- No role-based access changes to the tools (all signed-in users can use them).

4. Users & roles

The shell already models three roles (`admin` , `consultant` , `client`) via the mock auth. For the tool suite, any authenticated user can open any tool. Role gating can be layered on later per tool if needed.

5. The hub

`/home` shows module cards. Enabled cards link to their tool:

- **Pre PS, Time Auditor, Time Finder** (row 1)
- **Reason Eliminator, Power Planner, Meeting Framework, Post PS** (row 2)

6. The tools

6.1 Time Auditor (shell-native)

Time tracking & productivity analytics — the original base app. Routes: `/dashboard` , `/analytics` , `/time-auditor` , reports, challenges, etc.

6.2 Power Planner

Weekly planning tool. Users lay out commitments/goals ("results") and the actions under them, with a recurrence engine (weekly/monthly/quarterly/annually). Notable logic already built:

- A **"Repeat with goal"** ride-along: an action ticked under a recurring result repeats on **its own day** (gap-preserved before the result), not stacked on the result's day.
- Per-week edits to generated repeat copies **persist** (`userEdited` flag) instead of being overwritten on save.
- **"Hot refresh"** = a full data wipe via a reset flag (distinct from a normal refresh, which keeps data).

6.3 Reasons Eliminator

Assessment tool to surface and eliminate the "reasons"/excuses blocking a user, plus a power-word workflow and a grip-strength test/history. Multi-step flow with dashboard, previous assessments, and deep review.

6.4 Meeting Success Maximizer ("Meeting Framework")

Plan, run, and review meetings. Prep form, meeting list, meeting details, and an analytics dashboard. Exports to PDF/Excel (`jspdf` , `xlsx`).

6.5 Time Finder

Routine assessment & adjustment flow — select routines, set recurrence, align routines, find time savings, and review previous assessments.

7. Requirements

ID	Requirement	Status
P1	One login gates the whole suite	✅ (mock auth)
P2	Hub dashboard links to every tool	✅
P3	Each tool fully usable, data preserved	✅ (build green; browser QA pending)
P4	Consistent way back to hub from any tool	✅ (HubLink)
P5	Each tool keeps its own visual identity	✅ (per-tool theming)
P6	No data corruption between tools	✅ (namespaced storage)
P7	Deployable as static site	✅
P8	Backend-ready architecture	🕒 planned (see Backend Schema)

8. Open questions / future

- Should tools eventually **share** a user profile / data via the backend?
- Should the backend be Supabase, a custom Node API, or other? (Undecided — the frontend was deliberately kept backend-agnostic.)
- Per-tool role permissions?